

1. Bevezetés

1.1. A program célja

A „Legyen Ön is Milliomos” egy konzolos kvízzjáték, ami a TV-s műsor szabályait követi. A játékosnak egymás után 12 egyre nehezebb kérdésre kell válaszolnia. Minél tovább jut, annál több pénzt gyűjt össze, rossz válasz esetén viszont visszaesik a legutóbb elért biztos szintre.

1.1. Környezet

A program Linux konzolon fut, C++ nyelven készült.

Fordítás: `g++ -std=c++11 -Wall -Wextra -o loim main.cpp game.cpp highscore.cpp question.cpp filemanager.cpp`

1. Felépítés

1.1. Fileok

`main.cpp` - Főprogram

Itt van a `main()` belépési pont.

Létrehozza a `Game` objektumot.

`game.cpp` / `game.h` - Játékmenet vezérlése

A `Game` osztály kezeli a főmenüt, a játékmenetet, a segítségeket, a nyereményeket és a játékállapotot.

`question.cpp` / `question.h` - Kérdésoosztályok

A `Question` absztrakt ősoosztályból származik a `ChooseQuestion` és az `OrderQuestion`.

`filemanager.cpp` / `filemanager.h` - CSV betöltés

A `Filemanager` függvényei töltik be a kérdésfileokat.

`highscore.cpp` / `highscore.h` - Dicsőséglista

A `HighScoreEntry` egy rekordot, a `HighScoreTable` a teljes listát kezeli.

`colors.h` - Konzolszínek

1.1. Fordítási & Futtatási környezet

A program szabványos C++ könyvtárat használ.

Főbb elemek:

`std::string`, `vector`, `ifstream`, `ofstream`, `stringstream`, `mt19937`

A Konzolt ANSI escape kódokkal törli / színezi.

1. Adatszerkezetek

1.1. Alap konstansok

`Game::LEVELS = 12` - Összesen 12 szint van.

`Game::PRIZE_LADDER[LEVELS]` - a 12 lépcsős nyeremenylétra

Nem „technikailag” konstans, de a `main.cpp` / `highscore.h` fileban vannak a CSV fileok elérési path-jai.

Biztos szintek indexei: 4 és 9 (tehát 5. és 10. kérdés a biztos szint).

1.1. Feleletválasztós kérdés - ChooseQuestion

A `Question` osztályból származik.

A tárolt kérdés szövege, kategória, helyes válasz, négy válaszlehetőség.

Saját mező: `difficulty`, vagyis a nehézségi szint 1 és 12 között.

A kérdéseket szintenként vannak tárolva: `std::vector<ChooseQuestion>`
`chooseQuestions[LEVELS]`

1.1. Sorrendezős kérdés - OrderQuestion

Szintén a `Question` osztályból származik

4 elemet kell helyes sorrendbe rendezni, helyes válasz például: „ACBD” formájú

A kérdések egy `std::vector<OrderQuestion>` tárolóban vannak

1.1. Dicsőséglista elem – `HighScoreEntry`

Tárolt adatok: Játékos neve, dátum, nyeremény

A teljes listát a `HighScoreTable` kezeli

A bejegyzések `std::vector<HighScoreEntry>` tárolóban vannak

1. Fő működési folyamat

1.1. Indítás - `main.cpp`

Konzol inicializálása

A program létrehoz egy `Game` objektumot

Meghívja a kérdéseket betöltő függvényt

És elindítja a főmenü ciklust

1.1. Kérdésfileok betöltése

`FileManager::loadChooseQuestions()`

Megnyitja a feleletválasztós CSV filet

Átugorja a fejlécsort, „,” alapján darabolja a sorokat, pontosan 8 mezős sorokat dolgoz fel

Nehézségi szint alapján a kérdést beleteszi a megfelelő szint vektorába

`FileManager::loadOrderQuestions()`

Ugyanúgy feldolgozza a filet mint az előbb, csak itt 7 mezős sorokat dolgoz fel

A kérdéseket simán egy vektorban tároljuk mert itt nincs nehézségi szint

`FileManager::fileExists()`

A file létezését ellenőrzi

1.1. Főmenü (Main ciklus)

Menü elemek

[1] ÚJ JÁTÉK | Új játék loopot indít

[2] DICSŐSÉGLISTA | Kiírja a dicsőséglistát

[3] KILÉPÉS | Kilép

A menü választást a program egész számként olvassa be

Hibás bemenet esetén újra kérdez

1.1. Játékmenet - `Game::play()`

Nullázzuk az aktuális játékállapotot

Bekérjük a játékos nevét (max 20 karakter)

A sorrendező kérdések indexeit összekeveri

A 12 szinten végighalad amíg nincs rossz válasz vagy kiszállás

Játékmód almenü:

Csak feleletválasztós

Csak sorrendező

Vegyes (3. 7. 11. kérdés sorrendező)

Változók:

`hasChoose, hasOrder` | Létezik-e ilyen betöltött kérdés?

`currentLevel` | aktuális szint indexe (0 -> LEVELS-1)

`finalPrize` | végső nyeremény

`gameOver, walkAway` | játék végét indikáló változók

`used5050, usedAudience` | segítség már felhasználva?

1. Filekezelés és formátumok

1.1. Feleletválasztós file - /src/kerdesek.csv

Fejlécsor utáni formátum: Nehézség;Kérdés;A;B;C;D; Helyes;Kategória

1.1. Sorrendező file - /src/sorkerdesek.csv

Fejlécsor utáni formátum: Kérdés;A;B;C;D;Helyes;Kategória

1.1. Dicsőséglista - /src/dicsoseglista.csv

Fejlécsor utáni formátum: Név;Nyeremény;Dátum

HighScoreTable::load()	betölti a bejegyzéseket
HighScoreTable::save()	újraírja a teljes file-t, felülírja a betöltött vektorral
HighScoreTable::add()	új eredményt ad hozzá a betöltött vektorhoz
HighScoreTable::reset()	törli a listát, majd csak a fejléct menti vissza

1. Dinamikus memóriakezelés

A szövegek std::string objektumokban vannak

A válaszlehetőségek std::vector<std::string> tárolóban

A dicsőséglistát std::vector<HighScoreEntry> -ben

Ezeknek a felszabadítását STL tárolók destruktoraik automatikusan kezelik

1. Segédfüggvények & Logika

1.1. Konzol

Game::printSeparator()	elválasztó vonalat ír ki
Game::waitEnter()	enter lenyomására vár
Game::clearScreen()	ANSI escape kóddal törli a konzolt

A **color.h** szín escape kódokat tartalmaz

1.1. Biztonságos szintek

Game::isSafeLevel(int index)	igazat ad vissza ha az index 4 vagy 9
Game::getSafePrize(int index)	visszaadja az index szintje előtti legnagyobb biztos szint nyereményét

1.1. Segítségek (csak feleletválasztós kérdéseknél)

Game::apply5050()	Elrejt 2 helytelen válaszlehetőséget
Game::applyAudience()	A helyes válasz 30-60% közti értéket kap, a maradék %-ot a többi válaszlehetőség között szétosztjuk. Lehetséges, hogy nem a legnagyobb % a helyes.

1. Hibakezelés, Korlátok, Feltételezések

1.1. CSV hibák

A hibás mezőszámú sorokat a program átugorja

A hibás nehézségi szinttel rendelkező feleletválasztós kérdések nem kerülnek be a szintekhez

Ha egy adott játékmódhoz nincs kérdés, a program jelzi a problémát

1.1. Memóriakezelés

A program STL tárolókat és std::string objektumokat használ, ezért nincs kézi felszabadítás.

A destruktorkok és tárolók automatikusan kezelik az erőforrásokat.

1.1. Maximális elemszámok

Vektorok / Stringek használata miatt nincs limit

Oké technikailag $2^{63}-1$

A játékos neve legfeljebb 20 látható karakter lehet (kiírásnál így tudjuk szépen megjeleníteni)

1. *Továbbfejlesztési ötletek

Kérdésenkénti időkorlát

Nehézségi szint választása

Dicsőséglista bővítése játékidővel

Mivel az rng() egy seed-et generál, ezt a seed-et el lehetne menteni, és akár ugyanazon a kérdéssorozaton végigmenni / újrapróbálkozni vele